



Project Report

Intern Name: Ahmed Islam

Department: Artificial Intelligence

Organization: IR Solutions

Designation: AI Intern

Week # 08

Submitted to: Asad Ali

Submission Date: 29-Aug-2025

Table of Contents

Project Report	1
Theoretical Understanding.....	5
Image Classification.....	5
Convolutional Neural Networks (CNNs)	5
Overfitting and How to Prevent It.....	5
Solutions:	5
Using Pretrained Models	5
Rules of Machine Learning.....	6
Introduction	6
Key Concepts	6
1. Start Simple	6
2. Define Clear Metrics	6
3. Build Reliable Data Pipelines	6
4. Prevent Over-Engineering	6
5. Reuse Existing Knowledge	7
6. Monitor and Maintain	7
7. Keep Objectives Simple	7
8. Improve Through Iteration	7
Text Classification	7
Introduction	7
End-to-End Workflow	8
Gather Data	8
Explore Your Data	8
Choose a Model	8
Prepare Your Data	8
Build, Train, Evaluate	8
Tune Hyperparameters	8
Deploy the Model	9

Good Data Analysis	9
Key Techniques	9
Technical Foundations	9
Process & Approach	10
Essential Mindset	10
Deep Learning Tuning Playbook.....	11
Introduction	11
Target Audience	11
A Scientific Tuning Strategy	11
Key Components Covered	12
Training Pipeline and Run Configuration	12
Why It Matters	12
Introduction to Responsible AI	12
Introduction	12
What Is Responsible AI?	13
Fairness	13
Accountability	13
Safety.....	13
Privacy	13
Why It Matters	13
Adversarial Testing for Generative AI	13
Introduction	13
What Is Adversarial Testing?	13
Why It Matters	14
Adversarial Testing Workflow	14
Design adversarial queries	14
Evaluate model responses	14
Analyze potential risks	14
Implement mitigation	14

Inform deployment readiness	14
Self-directed Exploration	15
Git and GitHub	15
Diffusion Models	15
Modularizing Code for Speech-to-Speech Translation App	16
DevOps and Automation Basic & Tools	16
Conclusion	16
References	17

Theoretical Understanding

Image Classification

Image Classification means teaching a computer to recognize what is in a picture. For example, the model learns to say if a picture has a cat, a dog, or something else.

This is a **supervised learning task** because we train the model with labeled images (we already know what each image contains).

Convolutional Neural Networks (CNNs)

- A CNN is a special type of neural network built for images.
- **Convolution layers:** They scan small parts of an image (like looking at 3×3 blocks) to detect features such as edges, colors, or shapes.
- **Feature maps:** The results of these scans like a map that highlights where certain features (eyes, ears, etc.) appear.
- **Fully connected layers:** After features are extracted, these layers decide the final label (e.g., "dog").
- **Softmax:** The final function that gives probabilities for each possible class.

Overfitting and How to Prevent It

- **Overfitting:** When the model memorizes the training images but fails to perform well on new images.

Solutions:

- **Data Augmentation:** Slightly change the training images (rotate, flip, zoom) so the model learns to recognize objects in different conditions.
- **Dropout:** During training, randomly turn off some neurons so the model doesn't become too dependent on specific features.

Using Pretrained Models

- Training a CNN from scratch needs lots of data and time.
- Instead, we use **pretrained models** (already trained on millions of images).
- Two ways to use them:

- **Feature Extraction:** Use the knowledge (features) already learned by the model to classify new images.
- **Fine-Tuning:** Slightly adjust the pretrained model so it performs better on your specific dataset.

Rules of Machine Learning

Introduction

The "Rules of Machine Learning" by Google provide a set of practical guidelines for building machine learning systems effectively. These rules are based on real-world experience and emphasize that successful ML is less about complex algorithms and more about careful engineering, clear objectives, and robust data handling.

Key Concepts

1. Start Simple

- Do not rush into machine learning at the beginning.
- Use simple heuristics (rule-based systems) to get an initial working solution.
- This approach helps create a strong baseline and saves time before introducing complex models.

2. Define Clear Metrics

- Always establish measurable goals before building the system.
- Metrics help you evaluate whether the model is improving performance.
- Without the right metric, you might optimize the wrong thing and waste resources.

3. Build Reliable Data Pipelines

- A simple model with high-quality, consistent data usually performs better than a complex model with poor data.
- Ensure that your data pipelines are robust, tested, and consistent across training and deployment.

4. Prevent Over-Engineering

- Start with a basic model and improve it gradually.

- Validate both the pipeline and the model separately to avoid hidden issues such as missing features or corrupted inputs.

5. Reuse Existing Knowledge

- If you already have useful rules or domain knowledge, integrate them as features in your ML system.
- This ensures that prior insights are not lost when switching to machine learning.

6. Monitor and Maintain

- Continuously monitor data freshness and feature reliability.
- Silent failures (like missing or outdated features) can reduce model performance without being obvious.
- Assign ownership of feature columns to team members to maintain accountability.

7. Keep Objectives Simple

- Optimize a single, well-defined metric rather than juggling many indirect metrics.
- Direct metrics tied to user behavior (e.g., click-through rate) are more reliable than abstract ones (e.g., “user happiness”).

8. Improve Through Iteration

- Launch simple models quickly and keep improving them.
- Frequent iterations lead to better progress than waiting for a “perfect” solution.
- If the model stops improving, reconsider your objective may be misaligned.

Text Classification

Introduction

Text classification is a foundational machine learning task where the goal is to automatically assign labels to text. Whether sorting incoming emails into folders like “spam” or “important,” or analyzing user reviews for positive or negative sentiment, this workflow helps computers make sense of written content. Two main applications are topic classification (e.g., categorizing news articles by subject) and sentiment analysis (e.g., identifying whether a review is positive or negative).

End-to-End Workflow

The guide outlines a systematic, step-by-step workflow for tackling text classification with ML:

Gather Data

The quality and quantity of your training data directly impact model performance. Public datasets or APIs (like IMDb reviews, Twitter feeds, or news API) can be used, but you must aim for balanced representation across labels.

Explore Your Data

Understanding characteristics such as class balance, sample length, and word frequency helps tailor your approach. For example, IMDb reviews are evenly split between positive and negative labels; median words per sample are also noted to guide model choice.

Choose a Model

A special “step 2.5” helps determine the best model based on dataset traits. If your data has fewer words per sample relative to the number of samples, a Multi-Layer Perceptron (MLP) with n-gram features may work best. Recommendations are based on extensive testing across many datasets.

Prepare Your Data

Raw text must be tokenized and converted into numerical form (vectors). Two common methods:

- N-gram vectors (e.g., unigrams or bigrams) capture word occurrence in slices.
- Sequence embeddings model word order.

Techniques like feature selection and normalization are used to refine these representations

Build, Train, Evaluate

Using TensorFlow’s Keras API, models are assembled as layer stacks. You define the architecture, train the model, and assess its performance on validation data

Tune Hyperparameters

The initial model is just a starting point. Key hyperparameters like number of layers, units per layer, dropout rate, embedding size, and kernel size should be tweaked to improve performance

Deploy the Model

Once optimized, deploy your model (e.g., via Google Cloud). Ensure that production data matches the characteristics of your training data. Monitor performance and retrain when data drifts or degrades

Good Data Analysis

Introduction

Large, high-dimensional datasets such as logs or experimental records pose complex challenges that greatly differ from the manageable, small-scale data often seen in traditional scientific experiments. The Good Data Analysis guide by Google explores the practical strategies and mindset expert analysts use to draw reliable insights from such data, all while minimizing the risk of error and misinterpretation

Key Techniques

Technical Foundations

Effective data analysis begins with diving into the data's structure and behavior:

- Instead of relying solely on summary statistics like averages or medians, analysts explore full **distribution plots** (e.g., histograms) to detect patterns such as multiple peaks or significant outliers.
- **Outliers** are not just anomalies to discard; they may signal underlying issues, such as missing data or incorrect logging, and should be investigated rather than ignored.
- Even with large datasets, statistical noise can mislead conclusions. Thus, analysts include measures of confidence (e.g., confidence intervals, p-values) to express the reliability of their findings.
- Reviewing concrete data examples ensures that transformations and analysis code behave as expected a vital step in building trust in your results.
- Slicing data by meaningful subgroups (e.g., browser, locale, device type) helps detect inconsistencies across segments. Analysts must also watch for mix shifts—imbalances in group sizes that can distort comparisons.
- Analysts balance statistical significance with practical relevance, resisting the urge to overemphasize minor but statistically detectable differences that don't matter in real terms
- Checking data consistency over time—for example, day-over-day trends—helps catch shifts or anomalies early.

- Every filtering step applied to data should be clearly documented, including how much data is being excluded. Metrics like ratios must have well-defined numerators and denominators to avoid ambiguity.

Process & Approach

A disciplined analytical process sets the foundation for credible insights:

Analysts separate their work into three stages:

1. **Validation**—ensuring data accuracy and consistency,
2. **Description**—summarizing what the data actually shows, and
3. **Evaluation**—interpreting whether those findings benefit users or serve broader goals

Understanding the context of data collection such as experiment setup, instrumentation, or time is essential before any analysis begins.

Comparison of standard metrics (like user engagement) should precede examination of custom metrics, helping catch inconsistencies early.

Key insights should be measured multiple ways across different metrics or data sources to confirm reliability.

Analysts check for reproducibility across different samples and time frames; inconsistent results often indicate a fragile or flawed analysis.

When new metrics are developed, applying them to known data helps validate their reliability before interpreting new outcomes.

Analysis should be hypothesis-driven analysts form theories and then actively search for evidence that challenges (not just supports) those ideas.

For exploratory work, analysts favor iteration over perfection, getting quick, rough insights and refining as they go is more effective than striving for total precision from the start.

Analysts should avoid feedback loops where the metric used to evaluate success influences the system itself (e.g., more ads lead to more clicks) such loops can compromise the validity of conclusions.

Essential Mindset

The way analysts think and collaborate is as important as the tools they use:

- Good data analysis starts with questions, not data. Clear hypotheses guide meaningful investigation and prevent aimless work.

- Analysts act as both champions and skeptics, they explore interesting trends carefully, asking what might disprove their insights not just confirm them.
- The mantra “Correlation $\neq$ Causation” is central: analysts distinguish descriptive patterns from causal claims and communicate these boundaries clearly.
- Peer review is vital sharing analysis with colleagues first helps catch blind spots or assumptions that might otherwise go unnoticed.
- Embracing ignorance and mistakes with humility builds credibility; acknowledging limits and errors earns trust and fosters more honest analysis.

Deep Learning Tuning Playbook

Introduction

The Deep Learning Tuning Playbook is a practical guide created by experienced Google researchers and engineers that brings clarity to the often-obscure world of tuning deep neural networks. It emphasizes a scientific, methodical approach to model improvement, offering much-needed documentation around hyperparameter tuning, training pipelines, and debugging techniques. This guide fills a gap not typically addressed in academic papers or textbooks by revealing the "how-to" of deep learning performance optimization.

Target Audience

This playbook targets engineers and researchers who already have foundational knowledge in machine learning and deep learning. If you're still early in your ML journey, you're encouraged to begin with more introductory courses before applying the techniques in this guide.

A Scientific Tuning Strategy

At the heart of the playbook lies an **incremental and evidence-driven tuning strategy**. This process starts with a simple baseline model and, step by step, refines it by incorporating insights gained through experimentation:

1. **Set a clear goal** for each tuning round such as testing a regularizer or adjusting a specific hyperparameter.
2. **Design targeted experiments** to isolate and measure the effect of a single change.
3. **Evaluate results carefully**, ensuring that improvements are robust and reproducible, not just lucky anomalies.

4. **Adopt changes only when supported by solid evidence**, avoiding unnecessary complexity.

This disciplined approach emphasizes exploration understanding how changes affect performance over quick but shallow optimization of validation scores.

Key Components Covered

Guide for Starting a New Project

- Choosing the right model architecture, optimizer, batch size, and other initial configurations lays a strong foundation.
- Beginning with a solid baseline helps to clearly understand the impact of future improvements.

Training Pipeline and Run Configuration

- Advice includes how long to train (number of steps), depending on whether training is limited by compute resources or not.
- It also covers best practices for input pipeline optimization, experiment tracking, and checkpoint management.
- Additional attention is given to batch normalization nuances and maintaining pipelines across multi-host setups.

Why It Matters

Deep learning workflows often involve guesswork and trial-and-error this guide provides structure and repeatability. By formalizing tuning steps and pipeline checks, it helps practitioners avoid wasteful or erroneous practices, leading to faster, more stable improvements.

Introduction to Responsible AI

Introduction

As AI systems become part of our daily lives spanning domains such as healthcare, education, and business it's crucial to ensure their benefits far outweigh potential harms. The Introduction to Responsible AI guide from Google underscores these points, emphasizing ethical design and societal awareness when developing AI solutions.

What Is Responsible AI?

Responsible AI is the practice of designing, deploying, and scaling AI systems with a focus on four key ethical dimensions:

Fairness

Striving to ensure the AI does not produce biased or unjust outcomes across different groups.

Accountability

Establishing responsibility for AI decisions and being transparent about how they are made.

Safety

Ensuring that AI operates reliably without causing harm, even in unexpected scenarios.

Privacy

Protecting user data and controlling how personal information is managed and used.

These dimensions align with Google's broader AI Principles, which guide the development of technologies that are safe, trustworthy, and beneficial for all.

Why It Matters

Responsible AI is vital for preventing unintended consequences. When systems are developed without these considerations, they can inadvertently reinforce bias, violate privacy norms, or act unpredictably. Embracing a responsible framework helps build trust, ensure fairness, and safeguard societal well-being.

Adversarial Testing for Generative AI

Introduction

As generative AI systems become increasingly capable of producing text, images, and more, ensuring their safety and robustness is essential. The Adversarial Testing for Generative AI guide from Google provides a structured approach to "break" these models identifying weaknesses before they can harm users or produce unsafe content.

What Is Adversarial Testing?

Adversarial testing involves intentionally feeding models input designed to trigger problematic or unsafe output. Such inputs fall into two categories:

- **Explicitly adversarial queries:** These contain language or scenarios meant to directly provoke harmful, biased, or offensive responses.
- **Implicitly adversarial queries:** These seem harmless but involve sensitive or culturally charged topics like race, health, finance, or religion that may still lead to biased or concerning outputs.

Why It Matters

This type of testing is a proactive measure to expose potential failure modes before a system goes live. By understanding how a model responds to risky inputs, developers can implement targeted mitigations such as safeguards, filters, or further fine-tuning. It also helps inform decisions about whether the model is safe enough for public release or if more work is needed.

Adversarial Testing Workflow

The guide outlines a staged workflow to conduct adversarial testing effectively:

Design adversarial queries

Identify both clearly malicious inputs and subtle, culturally sensitive prompts that may lead to unsafe outputs.

Evaluate model responses

Observe how the model handles these adversarial prompts does it produce safe, neutral, or undesirable content?

Analyze potential risks

Categorize the types of failings ethical violations, policy breaches, or misinformation.

Implement mitigation

Based on findings, apply measures such as additional training, filtering rules, or policy enforcement layers to improve system safety.

Inform deployment readiness

Use insights from testing to assess whether the model is ready for deployment or needs further safeguarding

Self-directed Exploration

During this week, I dedicated time to self-directed exploration in order to strengthen my technical skills and gain hands-on knowledge in different areas. I focused on three key areas: Git & GitHub, Diffusion Models, and Modularizing Code for my speech-to-speech real-time translation web application.

Git and GitHub

I explored **Git**, a version control system, and **GitHub**, a platform that hosts Git repositories and enables collaboration. Git helps in tracking changes made to code, maintaining different versions, and reverting to previous states when needed. On the other hand, GitHub makes it easy to share projects with others, contribute to open-source repositories, and collaborate as a team.

Some useful Git commands I practiced include:

- **git init**: Initialize a new repository.
 - **git clone**: Copy an existing repository from GitHub.
 - **git status**: Check the current state of the working directory.
 - **git add .**: Stage changes for commit.
 - **git commit -m "message"**: Save changes with a message.
 - **git push**: Upload local changes to GitHub.
 - **git pull**: Fetch and merge changes from GitHub.
 - **git branch**: Manage different versions of a project.
 - **git checkout**: Switch between branches.
- These commands helped me understand how developers work in teams while ensuring project history and collaboration remain organized.

Diffusion Models

Next, I explored **Diffusion Models**, which are powerful techniques used in **generative AI** for creating images, audio, and even text. The main idea is to start with random noise and gradually transform it into meaningful data, like a realistic image, by reversing the noise process. The model is trained to learn how to remove noise step by step until the output resembles real-world data.

For example, in image generation, diffusion models can take pure random noise and iteratively refine it until a high-quality picture is produced. These models have become the backbone of many modern generative AI systems, such as DALL·E and Stable Diffusion. Understanding this concept gave me insights into how generative AI systems are built and why diffusion models are considered so effective in capturing details and producing high-quality outputs.

Modularizing Code for Speech-to-Speech Translation App

I am currently working on a **real-time speech-to-speech translation web app**, and as part of improving my project, I explored how to make the code **modularized**. Modularization means breaking the code into smaller, independent, and reusable parts (modules) instead of writing everything in a single file. This makes the project easier to maintain, debug, and expand in the future.

DevOps and Automation Basic & Tools

In addition, I explored the basics of **DevOps and automation**, focusing on the necessary tools that professionals use. I learned about:

- **Docker** – used for creating and running applications inside containers, ensuring consistency across environments.
- **Kubernetes** – an orchestration tool that helps manage multiple containers, handle scaling, and ensure smooth deployments.
- **Jenkins** – a popular automation server used for continuous integration and continuous delivery (CI/CD) pipelines.
- **Bitbucket/GitHub** – platforms for hosting code repositories and enabling collaboration.
- Other tools – like Ansible and Terraform, which are used for infrastructure automation and configuration management.

Exploring these tools gave me an understanding of how DevOps brings together development and operations, allowing faster delivery, automation of repetitive tasks, and reliable deployment of applications.

Conclusion

This report helped me strengthen my foundation in machine learning while exploring key practices like responsible AI, advanced testing, and optimization. Alongside, I gained

practical exposure to tools such as Git/GitHub, diffusion models, and modular coding through my speech-to-speech translation project. My exploration of DevOps and automation tools like Docker, Kubernetes, and Jenkins further broadened my understanding of scalable development. Overall, this journey combined theory with practice, preparing me to build reliable, ethical, and efficient AI systems.

References

<https://developers.google.com/machine-learning/guides/adv-testing>

<https://developers.google.com/machine-learning/guides/intro-responsible-ai>

<https://developers.google.com/machine-learning/guides/deep-learning-tuning-playbook>

<https://developers.google.com/machine-learning/guides/good-data-analysis>

<https://developers.google.com/machine-learning/guides/text-classification>

<https://developers.google.com/machine-learning/guides/rules-of-ml>

<https://developers.google.com/machine-learning/practica/image-classification>

Learn about the DevOps and Automation: https://softwareproductivity-my.sharepoint.com/:v:/g/personal/mubashir_naeem_spsnet_com/EZJkms4XwOlKtmKxo-uX69MBexeQv1OQFk8lbtP4_cls3A?e=gdyUHf&nav=eyJyZWZlcjJhbnEluZm8iOnsicmVmZXJ

Git Commands: https://softwareproductivity-my.sharepoint.com/personal/mubashir_naeem_spsnet_com/_layouts/15/stream.aspx?id=%2Fpersonal%2Fmubashir%5Fnaeem%5Fspsnet%5Fcom%2FDocuments%2FRecordings%2FSPS%20Internship%20Program%2C%20Summer%202025%20%2D%20DevOps%20%26%20Automation%20Track%2D20250723%5F150836%2DMeeting%20Recording%2Emp4&ga=1&referrer=StreamWebApp%2EWeb&referrerScenario=AddressBarCopied%2Eview%2E24ab83af%2D1479%2D493a%2D95ac%2D932abb261752

Diffusion Model: <https://www.youtube.com/watch?v=rI9VTiS99vw>